Introduction

In this section, you will learn about what Denix is, why it is needed, who can benefit from it, and popular practices for system configuration.

What is Denix

Denix is a Nix library designed for creating scalable configurations for NixOS, Home Manager, and Nix-Darwin.

It provides functions that transform input data into a module according to a specific algorithm. Thanks to this, if for any reason you need to create a module without using Denix, it will be sufficient to import the file with it, and everything will work.

The provided functions are generally divided into five categories:

- Creating configurations for Flake (NixOS, Home Manager, or Nix-Darwin)
- Options - an analogue of types and functions for creating options from Nixpkgs
- Modules
- Hosts
- Rices

Why and Who Needs Denix

Denix is primarily needed to simplify the creation, editing, and readability of configuration code. It eliminates the need to create typical expressions for your own modules, hosts, rices, etc.

If you plan to expand your configuration across multiple machines (hosts), want to have various system settings (rices) that can be changed with a single command, and strive to write readable and clean code, then you should consider trying Denix. Conversely, if you're creating a small configuration for a single machine and don't plan to expand it, Denix might be unnecessary.

Configuration templates using Denix can be found in the `templates` directory of the GitHub repository: github:yunfachi/denix?path=templates.

Modules

Custom modules are possibly the best practice for creating scalable configurations.

A module includes options, configuration, and importing other modules.

- `options = {};` : Similar to declaring variables in programming, but here it's a declaration of options.
- `config = {};` : Similar to initializing variables, but here it's about specifying values for options.
- `imports = [];` : A list of paths to modules or just module code (attrset or lambda that returns attrset).

NixOS, Home Manager, and Nix-Darwin have their own modules, which you have likely already worked with. You can search for options on these sites:

- NixOS: https://search.nixos.org/options
- Home Manager: https://home-manager-options.extranix.com/
- Nix-Darwin: https://nix-darwin.github.io/nix-darwin/manual/

Example of custom NixOS module code without using Denix:

nix

```nix
{
  lib,
  config,
  ...
```

```
}: {
  options = {
    example = {
      enable = lib.mkEnableOption "example" // {default = true;};
      hostnames = lib.mkOption {
        type = lib.types.listOf lib.types.str;
        default = [];
      };
    };
  };

  config = lib.mkIf (!config.example.enable) {
    networking.hosts."127.0.0.1" = config.example.hostnames;
  };
}
```

The same functionality but using Denix:

nix

```
{delib, ...}:
delib.module {
  name = "example";

  options.example = with delib; {
    enable = boolOption true;
    hostnames = listOfOption str [];
  };

  nixos.ifEnabled = {cfg, ...}: {
    networking.hosts."127.0.0.1" = cfg.hostnames;
  };
}
```

You can learn more about Denix modules in Modules.

Hosts

Host is any machine, such as a personal computer, server, etc.

The essence of this practice is to separate the configuration into a shared part and a unique part for each host.

Example of a host configuration module using Denix:

nix

```nix
{delib, ...}:
delib.host {
  name = "macbook";

  rice = "dark";
  type = "desktop";

  homeManagerSystem = "x86_64-darwin";
  home.home.stateVersion = "24.05";

  shared.myconfig = {
    services.openssh.authorizedKeys = ["ssh-ed25519 ..."];
  };
}
```

You can learn more about Denix hosts in Hosts.

Rices

Rice is a slang term used to describe system settings, especially related to appearance.

In our case, this is any configuration not tied to a specific host.

Example of a rice configuration module using Denix:

nix

```nix
{delib, ...}:
delib.rice {
  name = "dark";
```

```
    home.stylix = {
      polarity = "dark";
      colors = {
        # ...
      };
    };
  }
```

You can learn more about Denix rices in Rices.

Edit this page on Github

Last updated: 8/3/25, 7:32 AM

Next page
Initialization

Configuration Initialization

This section will describe creating the `minimal` template from scratch.

You do not have to do this; you can simply clone the minimal configuration template with the following command:

```sh
nix flake init -t github:yunfachi/denix#minimal
```

You can also clone the minimal configuration template without the rices:

```sh
nix flake init -t github:yunfachi/denix#minimal-no-rices
```

Flake

First, create a directory for your configuration and a `flake.nix` file with the following content:

flake.nix

```nix
{
  description = "Modular configuration of NixOS, Home Manager, and Nix-Da

  inputs = {
    nixpkgs.url = "github:nixos/nixpkgs/nixos-unstable";
    home-manager = {
      url = "github:nix-community/home-manager/master";
      inputs.nixpkgs.follows = "nixpkgs";
```

```
9       };
10      nix-darwin = {
11        url = "github:nix-darwin/nix-darwin/master";
12        inputs.nixpkgs.follows = "nixpkgs";
13      };
14      denix = {
15        url = "github:yunfachi/denix";
16        inputs.nixpkgs.follows = "nixpkgs";
17        inputs.home-manager.follows = "home-manager";
18        inputs.nix-darwin.follows = "nix-darwin";
19      };
20    };
21
22    outputs =
23      { denix, ... }@inputs:
24      let
25        mkConfigurations =
26          moduleSystem:
27          denix.lib.configurations {
28            inherit moduleSystem;
29            homeManagerUser = "sjohn"; # !!! REPLACEME
30
31            paths = [
32              ./hosts
33              ./modules
34              ./rices
35            ];
36
37            extensions = with denix.lib.extensions; [
38              args
39              (base.withConfig {
40                args.enable = true;
41              })
42            ];
43
44            specialArgs = {
45              inherit inputs;
46            };
47          };
48      in
```

```
49      {
50        # If you're not using NixOS, Home Manager, or Nix-Darwin,
51        # you can safely remove the corresponding lines below.
52        nixosConfigurations = mkConfigurations "nixos";
53        homeConfigurations = mkConfigurations "home";
54        darwinConfigurations = mkConfigurations "darwin";
55      };
56  }
```

If you are not familiar with `inputs` and `outputs`, read NixOS Wiki Flakes.

Code explanation:

- `mkConfigurations` - a function to reduce code repetition, which takes `moduleSystem` and passes it to `denix.lib.configurations`.
- `denix.lib.configurations` - Configurations (flakes) - Introduction.
- `paths = [./hosts ./modules ./rices];` - paths that will be recursively imported by Denix as modules. Remove `./rices` if you don't plan to use rices.
- `extensions` - Denix extensions that can add new functions, modules, and more (see Extensions - Introduction). For example, `args` simplifies adding arguments to modules, while `base` automatically generates template modules for host and rice options. The full list of official extensions and their configuration options can be found in All Extensions.

INFO

If you do not plan to use rices, add `rices.enable = false;` to the `base` extension configuration (after line 37).

TIP

You can also choose not to use any extensions at all - simply remove this argument. This tutorial supports that as well.

Hosts

Create a `hosts` directory, and within it, create a subdirectory with the name of your host, for example, `desktop`.

In this subdirectory, create a `default.nix` file with the following content:

nix

```nix
{delib, ...}:
delib.host {
  name = "desktop"; #!!! REPLACEME
}
```

In the same directory, create a `hardware.nix` file:

nix

```nix
{delib, ...}:
delib.host {
  name = "desktop"; #!!! REPLACEME

  homeManagerSystem = "x86_64-linux"; #!!! REPLACEME
  home.home.stateVersion = "24.05"; #!!! REPLACEME

  # If you're not using NixOS, you can remove this entire block.
  nixos = {
    nixpkgs.hostPlatform = "x86_64-linux"; #!!! REPLACEME
    system.stateVersion = "24.05"; #!!! REPLACEME

    # nixos-generate-config --show-hardware-config
    # other generated code here...
  };

  # If you're not using Nix-Darwin, you can remove this entire block.
  darwin = {
    nixpkgs.hostPlatform = "aarch64-darwin"; #!!! REPLACEME
    system.stateVersion = 6; #!!! REPLACEME
  };
}
```

The `default.nix` file will be modified later after adding modules and rices, so you can keep it open.

Rices

> **INFO**
>
> Skip this section if you do not wish to use rices.

Create a `rices` directory, and within it, create a subdirectory with the name of your rice, for example, `dark`.

In this subdirectory, create a `default.nix` file with the following content:

nix

```nix
{delib, ...}:
delib.rice {
  name = "dark"; #!!! REPLACEME
}
```

Modules

Create a `modules` directory, and within it, create a `config` subdirectory (typically, it contains modules that are not tied to a specific program or service).

It should be mentioned that modules represent your configuration, meaning it's up to your imagination, and you are free to change the modules as you wish.

Constants

In this subdirectory, create a `constants.nix` file with the following content:

nix

```nix
{delib, ...}:
delib.module {
  name = "constants";

  options.constants = with delib; {
```

```
    username = readOnly (strOption "sjohn"); #!!! REPLACEME
    userfullname = readOnly (strOption "John Smith"); #!!! REPLACEME
    useremail = readOnly (strOption "johnsmith@example.com"); #!!! REPLACEME
  };
}
```

This file is optional, as are any of its options, which are only used by you, but it is recommended as good practice.

Hosts

> **INFO**
>
> If you are using the `base` extension, do not create the `hosts.nix` file described below.

Also, create a `hosts.nix` file in this same directory (`modules/config`), and write any example from Hosts - Examples.

For example, we will take "With the `type` Option":

nix

```
{delib, ...}:
delib.module {
  name = "hosts";

  options = with delib; let
    host = {config, ...}: {
      options =
        hostSubmoduleOptions
        // {
          type = noDefault (enumOption ["desktop" "server"] null);

          isDesktop = boolOption (config.type == "desktop");
          isServer = boolOption (config.type == "server");
        };
    };
  in {
    host = hostOption host;
    hosts = hostsOption host;
```

```
  };

  home.always = {myconfig, ...}: {
    assertions = delib.hostNamesAssertions myconfig.hosts;
  };
}
```

If you added an example with new options (`type` , `displays` , etc.) or made your own options, don't forget to add values for these options in the hosts.

In our example, we added the `type` option, so open the `default.nix` file in your host's directory and add the attribute `type` to the `delib.host` function:

TIP

By default, the `base` extension also adds the `type` option, so you need to set it.

nix

```
{delib, ...}:
delib.host {
  name = "desktop"; #!!! REPLACEME

  type = "desktop" #!!! REPLACEME ["desktop"|"server"]

  # ...
}
```

Rices

INFO

Skip this section if you are not using rices.

INFO

If you are using the `base` extension, do not create the `rices.nix` file described below.

In the `modules/config` directory, create a `rices.nix` file, and write any example from Rices - Examples.

For example, we will take "Minimally Recommended Rice Module":

nix

```nix
delib.module {
  name = "rices";

  options = with delib; let
    rice = {
      options = riceSubmoduleOptions;
    };
  in {
    rice = riceOption rice;
    rices = ricesOption rice;
  };

  home.always = {myconfig, ...}: {
    assertions = delib.riceNamesAssertions myconfig.rices;
  };
}
```

Also, open the `default.nix` file of your host and add the attribute `rice` to the `delib.host` function:

nix

```nix
{delib, ...}:
delib.host {
  name = "desktop"; #!!! REPLACEME

  rice = "dark" #!!! REPLACEME

  # ...
}
```

Home Manager

If you created a constants module, just create a `home.nix` file with the following content:

nix

```nix
{delib, pkgs, ...}:
delib.module {
  name = "home";

  home.always = {myconfig, ...}: let
    inherit (myconfig.constants) username;
  in {
    home = {
      inherit username;
      # If you don't need Nix-Darwin, or if you're using it exclusively,
      # you can keep the string here instead of the condition.
      homeDirectory =
        if pkgs.stdenv.isDarwin
        then "/Users/${username}"
        else "/home/${username}";
    };
  };
}
```

If you did not use the constants module, the content of the file will be:

nix

```nix
{delib, pkgs, ...}:
delib.module {
  name = "home";

  home.always.home = {
    username = "sjohn"; #!!! REPLACEME
    # If you don't need Nix-Darwin, or if you're using it exclusively,
    # you can keep the string here instead of the condition.
    homeDirectory =
      if pkgs.stdenv.isDarwin
      then "/Users/sjohn" #!!! REPLACEME
      else "/home/sjohn"; #!!! REPLACEME
  };
}
```

User

You can also create a `user.nix` file with the configuration of your NixOS and Nix-Darwin user:

nix

```nix
{delib, ...}:
delib.module {
  name = "user";

  # If you're not using NixOS, you can remove this entire block.
  nixos.always = {myconfig, ...}: let
    inherit (myconfig.constants) username;
  in {
    users = {
      groups.${username} = {};

      users.${username} = {
        isNormalUser = true;
        initialPassword = username;
        extraGroups = ["wheel"];
      };
    };
  };

  # If you're not using Nix-Darwin, you can remove this entire block.
  darwin.always = {myconfig, ...}: let
    inherit (myconfig.constants) username;
  in {
    users.users.${username} = {
      name = username;
      home = "/Users/${username}";
    };
  };
}
```

If you did not use the constants module, the content of the file will be:

nix

```nix
{delib, ...}:
```

```
delib.module {
  name = "user";

  # If you're not using NixOS, you can remove this entire block.
  nixos.always.users = {
    groups.sjohn = {}; #!!! REPLACEME

    users.sjohn = { #!!! REPLACEME
      isNormalUser = true;
      initialPassword = "sjohn"; #!!! REPLACEME
      extraGroups = ["wheel"];
    };
  };

  # If you're not using Nix-Darwin, you can remove this entire block.
  darwin.always.users.users."sjohn" = { #!!! REPLACEME
    name = "sjohn"; #!!! REPLACEME
    home = "/Users/sjohn"; #!!! REPLACEME
  };
}
```

Conclusion

If you have followed the instructions precisely, you will end up with the following configuration directory tree:

plaintext

```
hosts
- desktop
  - default.nix
  - hardware.nix
modules
- config
  - constants.nix
  - home.nix
  - hosts.nix (if you have not used the `base` extension)
  - rices.nix (if you have not used the `base` extension)
```

```
  - user.nix
rices (if you have used rices)
- dark
  - default.nix
flake.nix
```

You can check if everything is set up correctly using the command:

```sh
nix flake check .#
```

Edit this page on Github

Last updated: 7/18/25, 10:25 AM

Previous page
Introduction

Next page
First Modules

First Modules

In this section, we will create modules for some programs. To start, create the `programs` and `services` subdirectories in the `modules` directory.

Creating your own modules is almost the same as creating regular NixOS modules, so you can look for NixOS options here, Home Manager options here, and Nix-Darwin options here.

Git

Let's assume you already have a constants module. Create a file `modules/programs/git.nix` with the following content:

nix

```nix
{delib, ...}:
delib.module {
  name = "programs.git";

  options.programs.git = with delib; {
    enable = boolOption true;
    enableLFS = boolOption true;
  };

  home.ifEnabled.programs.git = {myconfig, cfg, ...}: {
    enable = cfg.enable;
    lfs.enable = cfg.enableLFS;

    userName = myconfig.constants.username;
    userEmail = myconfig.constants.useremail;
  };
```

```
}
```

Code Explanation:

- `enable` - an option to enable or disable the module entirely.
- `enableLFS` - an option to enable Git Large File Storage.

Hyprland

Assume that your host has the options `type` and `isDesktop`. Create a subdirectory `hyprland` in the `modules/programs/` directory, and in it, create a file `default.nix` with the following content:

nix

```nix
{delib, ...}:
delib.module {
  name = "programs.hyprland";

  options = {myconfig, ...} @ args: delib.singleEnableOption myconfig.host.is

  nixos.ifEnabled.programs.hyprland.enable = true;
  home.ifEnabled.wayland.windowManager.hyprland.enable = true;
}
```

Also, create a file `settings.nix` in this directory:

nix

```nix
{delib, ...}:
delib.module {
  name = "programs.hyprland";

  home.ifEnabled.wayland.windowManager.hyprland.settings = {
    "$mod" = "SUPER";

    general = {
      gaps_in = 5;
```

```
      gaps_out = 10;
    };
  };
}
```

And finally, the file `binds.nix` :

nix

```nix
{delib, ...}:
delib.module {
  name = "programs.hyprland";

  home.ifEnabled.wayland.windowManager.hyprland.settings = {
    bindm = [
      "$mod, mouse:272, movewindow"
      "$mod, mouse:273, resizewindow"
    ];

    bind = [
      "$mod, q, killactive,"
      "CTRLALT, Delete, exit,"

      "$mod, Return, exec, kitty" # your terminal
    ];
  };
}
```

We have created a small configuration for Hyprland, which you can expand and add new options to as needed.

Edit this page on Github

Last updated: 5/20/25, 11:28 AM

Previous page
Initialization

Next page
Transfer to Denix

Transfer to Denix

If you already have a NixOS, Home Manager, or Nix-Darwin configuration, you can transfer most of the code without significant changes and then adapt it for Denix.

However, you will need to create the following from scratch:

- Hosts
- Rices (if you want)
- Some initial modules

The main part of the configuration can be fully reused from your old setup. The key is to separate the hardware configuration from the general configuration. See the section How Does It Work?.

How Does It Work?

All Denix modules are standard NixOS, Home Manager, or Nix-Darwin modules but with additional logic for enabling and disabling configurations.

This means that you can add code or files from the old configuration into the new one, so they are imported through `delib.configurations` . You can place this code in the `modules` directory or create a new one, for example, `modules_nixos_old` for older configurations.

Example of a Simple Configuration

Suppose you have an old configuration consisting of two files: `configuration.nix` and `hardware-configuration.nix`, and you've already created hosts and the `modules/config` directory following the instructions. The configuration for your host should include `hardware-configuration.nix`, so all that remains is to copy `configuration.nix` into the `modules` directory and remove unnecessary options, like `system.stateVersion`.

Example of a Complex Configuration

Suppose you have an old configuration with hosts and multiple modules split across files. Hosts are typically specific to your system, so you will need to transfer them manually, usually by just copying the code.

Modules (e.g., for programs and services) can simply be copied into the `modules` directory or other files imported via `delib.configurations`.

Edit this page on Github

Last updated: 5/23/25, 9:10 AM

Previous page
First Modules

Next page
Introduction to NixOS Modules

Introduction to NixOS Modules

A NixOS module is a file containing a Nix expression with a specific structure. It defines options (options) and values for those options (config). For example, the `/etc/nixos/configuration.nix` file is also a module.

Home Manager and Nix-Darwin modules work similarly, but they can be used not only in NixOS but also on other systems.

This is not a comprehensive guide on Nix modules, so it is recommended to read the NixOS Wiki article on modules.

Structure

The structure of a NixOS module:

nix

```nix
{
  imports = [
    # Paths to other modules or Nix expressions.
  ];

  options = {
    # Declaration of options.
  };

  config = {
    # Assigning values to previously declared options.
    # For example, networking.hostName = "denix";
  };
```

```
  # However, values are usually assigned to options not in config, but here.
  # For example, networking.hostName = "denix";
}
```

Function

A module can also be a function (lambda) that returns an attribute set:

nix

```nix
{lib, ...} @ args: {
  # Formally, this is config._module.args.hostname
  _module.args.hostname = lib.concatStrings ["de" "nix"];

  imports = [
    {hostname, config, ...}: {
      config = lib.mkIf config.customOption.enable {
        networking.hostName = hostname;
      };
    }
  ];
}
```

Imports

`imports` is a list of relative and absolute paths, as well as functions and attribute sets:

nix

```nix
{
  imports = [
    ./pureModule.nix
    /etc/nixos/impureModule.nix
    {pkgs, ...}: {
      # ...
    }
  ];
}
```

Nixpkgs Options

Options are usually declared using `lib.mkOption` :

```nix
optionName = lib.mkOption {
  # ...
};
```

`lib.mkOption` accepts an attribute set. Some attributes include:

- `type` : the type of value for this option, e.g., `lib.types.listOf lib.types.port` .
- `default` : the default value for this option.
- `example` : an example value for this option, used in documentation.
- `description` : a description of this option, also used in documentation.
- `readOnly` : whether the option can be modified after declaration.

For more information on Nixpkgs options, see the NixOS Wiki.

Denix Options

Denix uses a different approach to options, although both methods can be used simultaneously.

An example of a module with options using Denix:

```nix
{delib, ...}:
delib.module {
  name = "coolmodule";

  options.coolmodule = with delib; {
    # boolOption <default>
    enable = boolOption false;
    # noDefault (listOf) <default>
    ports = noDefault (listOfOption port null);
    # allowNull (strOption) <default>
    email = allowNull (strOption null);
```

```
  };
}
```

For more information on Denix options, see the Options section.

Creating Your Own Modules (Options)

Declaring your own options is a great practice if you want to enable or disable certain parts of the code. This can be useful if you have multiple hosts (machines) with the same configuration, and, for example, machine `foo` does not need a program that is used on machine `bar`.

An example of a NixOS module for simple git configuration:

nix

```nix
{lib, config, ...}: {
  options.myconfig.programs.git = {
    enable = lib.mkEnableOption "git" // {default = true;};
  };

  config = lib.mkIf config.myconfig.programs.git.enable {
    programs.git = {
      enable = true;
      lfs.enable = true;

      config = {
        init.defaultBranch = "master";
      };
    };
  };
}
```

The same configuration but using Denix:

nix

```nix
{delib, ...}:
delib.module {
```

```
  name = "programs.git";

  options = delib.singleEnableOption true;

  nixos.ifEnabled.programs.git = {
    enable = true;
    lfs.enable = true;

    config = {
      init.defaultBranch = "master";
    };
  };
}
```

Edit this page on Github

Last updated: 8/17/25, 4:37 PM

Previous page
Transfer to Denix

Next page
Introduction

Denix Documentation

Menu On this page

Introduction to Denix Modules

A Denix module is the same as a NixOS, Home Manager, or Nix-Darwin module, but its attribute set is wrapped in the `delib.module` function.

No Limitations

This means that you can use all three types of modules simultaneously, although it's unlikely you'll need anything other than the first option:

Denix Module

nix

```nix
{delib, ...}:
delib.module {
  name = "...";
}
```

Denix Module with NixOS/Home Manager/Nix-Darwin Module

nix

```nix
{delib, ...}:
delib.module {
  name = "...";
} // {

}
```

NixOS/Home Manager/Nix-Darwin Module Only

nix

```
{

}
```

Simplicity and Cleanliness

Denix modules tend to look simpler and cleaner compared to NixOS/Home Manager/Nix-Darwin modules, due to the following reasons:

1. Simple yet fully functional option declaration (see Options).
2. Built-in logic for separating configurations based on the value of `${delib.module :: name}.enable` : always, ifEnabled, ifDisabled.
3. Shared options but separated configurations for NixOS, Home Manager, Nix-Darwin, and custom options.

Edit this page on Github

Last updated: 5/23/25, 9:10 AM

Previous page
Introduction to NixOS Modules

Next page
Structure

Denix Documentation

Menu On this page

Structure

This section uses the variables `myconfigName` , `moduleSystem` , and `homeManagerUser` , so it's recommended to first review the corresponding subsection: delib.configurations Function Arguments.

Function Arguments

- `name` : string. It is recommended that it matches the parent path of the module's `enable` option, such as `"programs.example"` , since the `cfg` argument depends on this path.
- `options` : attrset or a lambda that returns an attrset (see Options).
- `[myconfig|nixos|home|darwin].[ifEnabled|ifDisabled|always]` : attrset or a lambda that returns an attrset.
  - `myconfig.*` : sets values in `config.${myconfigName}` .
  - `nixos.*` : sets values in `config` if `moduleSystem` is `nixos` . Otherwise, does nothing.
  - `home.*` : sets values in `config` if `ModuleSystem` is `home` . Otherwise, sets it in `config.home-manager.users.${homeManagerUser}` .
  - `darwin.*` : sets values in `config` if `moduleSystem` is `darwin` . Otherwise, does nothing.
  - `[myconfig|nixos|home|darwin].ifEnabled` : executed only if the `cfg.enable` option (see Passed Arguments - cfg) exists and is `true` .
  - `[myconfig|nixos|home|darwin].ifDisabled` : executed only if the `cfg.enable` option exists and is `false` .
  - `[myconfig|nixos|home|darwin].always` : always executed, even if the `cfg.enable` option doesn't exist.

Passed Arguments

A list of arguments passed to `options` and `[myconfig|nixos|home|darwin].[ifEnabled|ifDisabled|always]`, if their type is a `lambda`:

- `name`: the same name from the `delib.module` arguments.
- `myconfig`: equals `config.${myConfigName}`.
- `cfg`: essentially, the config values (assigned options) of the current module. In other words, equals `config.${myConfigName}.${name}`, where `name` is the argument from `delib.module` and is "expanded" (converted to a valid attribute path).
- `parent`: equal to the "parent" module (attribute set) of `cfg`. Example: `parent` is `config.${myConfigName}.programs` if `cfg` is `config.${myConfigName}.programs.example`.

Pseudocode

nix

```nix
delib.module {
  name = "";

  # options.${myConfigName} = ...
  options = {name, cfg, parent, myconfig, ...}: {};

  # config.${myConfigName} = ...
  myconfig.ifEnabled = {name, cfg, parent, myconfig, ...}: {};
  myconfig.ifDisabled = {name, cfg, parent, myconfig, ...}: {};
  myconfig.always = {name, cfg, parent, myconfig, ...}: {};

  # if moduleSystem == "nixos"
  # then {config = ...;}
  # else {}
  nixos.ifEnabled = {name, cfg, parent, myconfig, ...}: {};
  nixos.ifDisabled = {name, cfg, parent, myconfig, ...}: {};
  nixos.always = {name, cfg, parent, myconfig, ...}: {};
```

```
  # if moduleSystem == "home"
  # then {config = ...;}
  # else {config.home-manager.users.${homeManagerUser} = ...;}
  home.ifEnabled = {name, cfg, parent, myconfig, ...}: {};
  home.ifDisabled = {name, cfg, parent, myconfig, ...}: {};
  home.always = {name, cfg, parent, myconfig, ...}: {};

  # if moduleSystem == "darwin"
  # then {config = ...;}
  # else {}
  darwin.ifEnabled = {name, cfg, parent, myconfig, ...}: {};
  darwin.ifDisabled = {name, cfg, parent, myconfig, ...}: {};
  darwin.always = {name, cfg, parent, myconfig, ...}: {};
}
```

Edit this page on Github

Last updated: 8/29/25, 5:59 PM

Previous page
Introduction

Next page
Examples

Denix Documentation

Menu On this page

Examples

Constants

nix

```nix
{delib, ...}:
delib.module {
  name = "constants";

  options.constants = with delib; {
    username = readOnly (strOption "sjohn");
    userfullname = readOnly (strOption "John Smith");
    useremail = readOnly (strOption "johnsmith@example.com");
  };
}
```

Home Manager

With constants:

nix

```nix
{delib, pkgs, ...}:
delib.module {
  name = "home";

  home.always = {myconfig, ...}: let
    inherit (myconfig.constants) username;
  in {
    home = {
```

```
      inherit username;
      # If you don't need Nix-Darwin, or if you're using it exclusively,
      # you can keep the string here instead of the condition.
      homeDirectory =
        if pkgs.stdenv.isDarwin
        then "/Users/${username}"
        else "/home/${username}";
    };
  };
}
```

User

With constants:

nix

```
{delib, ...}:
delib.module {
  name = "user";

  # If you're not using NixOS, you can remove this entire block.
  nixos.always = {myconfig, ...}: let
    inherit (myconfig.constants) username;
  in {
    users = {
      groups.${username} = {};

      users.${username} = {
        isNormalUser = true;
        initialPassword = username;
        extraGroups = ["wheel"];
      };
    };
  };

  # If you're not using Nix-Darwin, you can remove this entire block.
  darwin.always = {myconfig, ...}: let
```

```
    inherit (myconfig.constants) username;
  in {
    users.users.${username} = {
      name = username;
      home = "/Users/${username}";
    };
  };
}
```

Git

With constants:

nix

```
{delib, ...}:
delib.module {
  name = "programs.git";

  options.programs.git = with delib; {
    enable = boolOption true;
    enableLFS = boolOption true;
  };

  home.ifEnabled.programs.git = {myconfig, cfg, ...}: {
    enable = true;
    lfs.enable = cfg.enableLFS;

    userName = myconfig.constants.username;
    userEmail = myconfig.constants.useremail;
  };
}
```

Alejandra

nix

```nix
{delib, pkgs, ...}:
delib.module {
  name = "programs.alejandra";

  options = delib.singleEnableOption true;

  home.ifEnabled.home.packages = [pkgs.alejandra];
}
```

Edit this page on Github

Last updated: 6/6/25, 8:42 PM

Previous page
Structure

Next page
Introduction

Denix Documentation

Menu On this page

Introduction to Denix Options

Denix options are a wrapper for `lib.mkOption`. This means that any Denix option can be created using the standard `lib.mkOption`, and using Denix options is optional.

Why use Denix options?

Using `lib.mkOption` can be cumbersome, and writing something like this every time:

```nix
lib.mkOption {type = lib.types.listOf lib.types.str; default = [];}
```

is inconvenient, especially when options are used extensively in your configuration. Instead, you can write something more concise:

```nix
delib.listOfOption delib.str []
```

Thus, instead of creating an option by specifying all parameters, you can use Denix functions for more readable and cleaner code.

Working Principle

Functions related to options are divided into four main types:

Creating an Option with Type N

- `*Option <default>` - for example, `strOption "foo"` .
- `*OfOption <secondType> <default>` - for example, `listOfOption str ["foo" "bar"]` .
- `*ToOption <secondType> <default>` - for example, `lambdaToOption int (x: 123)` .

Adding Type N to Option Type X

- `allow* <option>` - for example, `allowStr ...` .
- `allow*Of <secondType> <option>` - for example, `allowListOf str ...` .
- `allow*To <secondType> <option>` - for example, `allowLambdaTo int ...` .

Directly Modifying the Attribute Set of an Option

- `noDefault <option>` - removes the `default` attribute from the option. It is rarely used since options without a default value are uncommon.
- `noNullDefault <option>` - removes the `default` attribute from the option if its value is `null` . Used for default value logic where its absence is allowed.
- `readOnly <option>` - adds the `readOnly` attribute with the value `true` .
- `apply <option> <apply>` - adds the `apply` attribute with the value passed to this function.
- `description <option> <description>` - adds the `description` attribute with the given value.

Generating Specific Options

- `singleEnableOption <default> { name, ... }` - creates an attribute set using the expression `delib.setAttrByStrPath "${name}.enable" (boolOption default)` . This is commonly used in `delib.module :: options` with its passed arguments, so you can simply write:

```nix
options = delib.singleEnableOption <default>;
```

- `singleCascadeEnableOption { parent, ... }` - creates an attribute set via `singleEnableOption` above, but uses `parent.enable` as the `<default>` . See the

example:

nix

```nix
# if current module's name is "programs.category.example",
# enable it by default if "programs.category" is enabled.
# will error if "programs.category.enable" is missing.
options = delib.singleCascadeEnableOption;
```

- `moduleOptions <opts> <args>` - creates an attribute set with `<opts>` for the current module path. Usually, you need to use the module `name` once again to define all options, but this wrapper does this automatically. `<args>` is a set with passed arguments. If `<opts>` is a function, then `<args>` set is automatically passed to it. See the example:

nix

```nix
# current module's name is "programs.category.example".

# if `singleEnableOption` is not enough:
options = with delib; moduleOptions {
  enable = boolOption true;
  device = strOption "desktop";
};

# `options` block above is the same as:
options.programs.category.example = with delib; {
  enable = boolOption true;
  device = strOption "desktop";
};

# but if `singleCascadeEnableOption` is not enough:
options = with delib; moduleOptions ({ parent, ... }: {
  enable = boolOption parent.enable;
  device = strOption "desktop";
}); # notice the parentheses

# once again, `options` block above is the same as:
options = with delib; { parent, ... }: {
  programs.category.example = {
    enable = boolOption parent.enable;
    device = strOption "desktop";
```

```
    };
  };
```

The list of current options can be found in the source code: github:yunfachi/denix?path=lib/options.nix

---

Examples

Denix	lib.mkOption
`portOption 22`	`mkOption {type = types.port; default = 22;}`
`noDefault (portOption null)`	`mkOption {type = types.port;}`
`noNullDefault (if myconfig.server.enable then "127.0.0.1:4533" else null)`	`mkOption {type = types.str;} // (if myconfig.server.enable then {default = "127.0.0.1:4533";} else {})`
`allowNull (portOption null)`	`mkOption {type = types.nullOr types.port; default = null;}`
`allowStr (portOption "22")`	`mkOption {type = types.strOr types.port; default = "22";}`
`listOfOption port []`	`mkOption {type = types.listOf types.port; default = [];}`
`readOnly (noDefault (portOption null))`	`mkOption {type = types.port; readOnly = true;}`
`singleEnableOption true {name = "git";}`	`git.enable = mkEnableOption "git" // {default = true;}`

> *(Usually `{name = "git";}` is not required, as this function is mainly used in `delib.module :: options`)*

Edit this page on Github

Last updated: 8/31/25, 12:01 PM

Previous page
Examples

Next page
Introduction

Introduction to Denix Hosts

A Denix host is an attribute set returned by the function `delib.host`, which enables or disables specific configurations depending on the value of the option `${myconfigName}.host`. It also passes the input attribute set of the `delib.host` function to the `${myconfigName}.host.${delib.host :: name}` option.

In simple terms, this allows you to separate the NixOS, Home Manager, Nix-Darwin, and custom options configuration into those that apply only to the current host and those that apply to all hosts. For example, the former can be used to enable or disable certain programs, while the latter can be used to add the current host's SSH key to the `authorizedKeys` of all hosts.

A host can also be compared to a module, because all hosts are imported regardless of which one is active, but the applied configurations depend on the active host.

It is also worth mentioning that, by default, the Home Manager module is automatically included in the NixOS and Nix-Darwin configurations of each host. If you do not want to use the Home Manager module for a particular host, simply add `useHomeManagerModule = false;` to the arguments of the `delib.host` function.

Options

For hosts to work, the configuration must include the options `${myconfigName}.host` and `${myconfigName}.hosts`, which **you define yourself** in one of the modules.

Here is an example of a minimal recommended host configuration:

nix

```nix
{delib, ...}:
```

```
delib.module {
  name = "hosts";

  options = with delib; let
    host = {
      options = hostSubmoduleOptions;
    };
  in {
    host = hostOption host;
    hosts = hostsOption host;
  };

  home.always = {myconfig, ...}: {
    assertions = delib.hostNamesAssertions myconfig.hosts;
  };
}
```

More examples can be found in the Examples section.

Edit this page on Github

Last updated: 6/21/25, 5:17 PM

Previous page
Introduction

Next page
Structure

Introduction to Denix Configurations (Flakes)

The `delib.configurations` function is used to create lists of `nixosConfigurations`, `homeConfigurations`, and `darwinConfigurations` for flakes.

In addition to all hosts, it also adds combinations of each host with every **non-`inheritanceOnly`** rice, which allows for quickly switching between rice configurations without editing the code. For example, if the "desktop" host is set to use the "light" rice, executing the following command:

```sh
nixos-rebuild switch --flake .#desktop --use-remote-sudo
```

will use the "desktop" host with the "light" rice. However, if you need to quickly switch to another rice, for example, "dark" you can run the following command:

```sh
nixos-rebuild switch --flake .#desktop-dark --use-remote-sudo
```

In this case, the host remains "desktop", but the rice changes to "dark".

It is important to note that when switching rice in this way, only the value of the `${myConfigName}.rice` option changes, while the value of `${myConfigName}.hosts.${hostName}.rice` remains the same.

Principle of Configuration List Generation

The configuration list is generated based on the following principle:

- `{hostName}` - where `hostName` is the name of any host.

- `{hostName}-{riceName}` - where `hostName` is the name of any host, and `riceName` is the name of any rice where `inheritanceOnly` is `false`.

If `moduleSystem` from the function arguments is set to `home`, then a prefix of `{homeManagerUser}@` is added to all configurations in the list.

Example

An example of a flake's `outputs` for `nixosConfigurations`, `homeConfigurations`, and `darwinConfigurations`:

nix

```nix
outputs = {denix, ...} @ inputs: let
  mkConfigurations = moduleSystem:
    denix.lib.configurations rec {
      inherit moduleSystem;
      homeManagerUser = "sjohn";

      paths = [./hosts ./modules ./rices];

      specialArgs = {
        inherit inputs moduleSystem homeManagerUser;
      };
    };
in {
  nixosConfigurations = mkConfigurations "nixos";
  homeConfigurations = mkConfigurations "home";
  darwinConfigurations = mkConfigurations "darwin";
}
```

Using multiple channels

To conveniently work with multiple channels in the configuration, the `delib.configurations` function accepts the arguments `nixpkgs`, `home-manager`, and

`nix-darwin` , which are used in the generated configurations.

By default, there is no need to specify these arguments - they default to the corresponding values from the Denix flake's `inputs` . These can also be overridden via `inputs.denix.inputs.<input>.follows` , which is more convenient for users who rely on a single channel.

Therefore:

- `delib.configurations :: [nixpkgs, home-manager, nix-darwin]` - for those who need multiple channels.
- `inputs.denix.<nixpkgs|home-manager|nix-darwin>.follows` - for those who only need a single channel.

Based on this, several usage options are possible:

1. Override `nixpkgs` , `home-manager` , and `nix-darwin` via the arguments of the `delib.configurations` function for both "stable" and "unstable" channels used in `./hosts/stable` and `./hosts/unstable` , respectively. Note that the implementation of the `mkConfigurations` function can vary - this is just an example.

nix

```nix
{
  inputs = {
    nixpkgs-unstable.url = "github:nixos/nixpkgs/nixos-unstable";
    nixpkgs-stable.url = "github:nixos/nixpkgs/nixos-25.05";

    home-manager-unstable = {
      url = "github:nix-community/home-manager/master";
      inputs.nixpkgs.follows = "nixpkgs-unstable";
    };
    home-manager-stable = {
      url = "github:nix-community/home-manager/release-25.05";
      inputs.nixpkgs.follows = "nixpkgs-stable";
    };

    nix-darwin-unstable = {
      url = "github:nix-darwin/nix-darwin/master";
      inputs.nixpkgs.follows = "nixpkgs-unstable";
    };
```

```
    nix-darwin-stable = {
      url = "github:nix-darwin/nix-darwin/nix-darwin-25.05";
      inputs.nixpkgs.follows = "nixpkgs-stable";
    };

    denix.url = "github:yunfachi/denix";
  };

  outputs = {
    denix,
    nixpkgs-unstable,
    nixpkgs-stable,
    home-manager-unstable,
    home-manager-stable,
    nix-darwin-unstable,
    nix-darwin-stable,
    ...
  } @ inputs: let
    _mkConfigurations = nixpkgs: home-manager: nix-darwin: hosts: moduleSyste
      denix.lib.configurations {
        inherit moduleSystem nixpkgs home-manager nix-darwin;
        homeManagerUser = "sjohn";

        paths = [./modules ./rices] ++ hosts;

        specialArgs = {
          inherit inputs;
        };
      };

    mkConfigurations = moduleSystem:
      nixpkgs-unstable.lib.attrsets.mergeAttrsList
      (map (f: f moduleSystem) [
        (_mkConfigurations nixpkgs-stable home-manager-stable nix-darwin-stab
        (_mkConfigurations nixpkgs-unstable home-manager-unstable nix-darwin-
      ]);
  in {
    nixosConfigurations = mkConfigurations "nixos";
    homeConfigurations = mkConfigurations "home";
    darwinConfigurations = mkConfigurations "darwin";
```

```
  };
}
```

2. Override `nixpkgs`, `home-manager`, and `nix-darwin` using `inputs.denix.inputs.<name>.follows`. This is the recommended method for those using a single channel.

nix

```nix
{
  inputs = {
    nixpkgs.url = "github:nixos/nixpkgs/nixos-unstable";

    home-manager = {
      url = "github:nix-community/home-manager/master";
      inputs.nixpkgs.follows = "nixpkgs";
    };

    nix-darwin = {
      url = "github:nix-darwin/nix-darwin/master";
      inputs.nixpkgs.follows = "nixpkgs";
    };

    denix = {
      url = "github:yunfachi/denix";
      inputs.nixpkgs.follows = "nixpkgs";
      inputs.home-manager.follows = "home-manager";
      inputs.nix-darwin.follows = "nix-darwin";
    };
  };

  outputs = {denix, ...} @ inputs: let
    mkConfigurations = moduleSystem:
      denix.lib.configurations {
        inherit moduleSystem;
        homeManagerUser = "sjohn";

        paths = [./hosts ./modules ./rices];

        specialArgs = {
          inherit inputs;
```

```
        };
      };
  in {
    nixosConfigurations = mkConfigurations "nixos";
    homeConfigurations = mkConfigurations "home";
    darwinConfigurations = mkConfigurations "darwin";
  };
}
```

3. Do not override anything. Also suitable for single-channel use, though less recommended.

nix

```
{
  inputs = {
    denix.url = "github:yunfachi/denix";
  };

  outputs = {denix, ...} @ inputs: let
    mkConfigurations = moduleSystem:
      denix.lib.configurations {
        inherit moduleSystem;
        homeManagerUser = "sjohn";

        paths = [./hosts ./modules ./rices];

        specialArgs = {
          inherit inputs;
        };
      };
  in {
    nixosConfigurations = mkConfigurations "nixos";
    homeConfigurations = mkConfigurations "home";
    darwinConfigurations = mkConfigurations "darwin";
  };
}
```

Edit this page on Github

Last updated: 7/8/25, 1:07 PM

Previous page
Examples

Next page
Structure

Structure

Function Arguments

- `myconfigName` (string): the category for all Denix module options, hosts, and rices. Defaults to `myconfig` ; changes are not recommended.
- `denixLibName` (string): the name of the Denix library in `specialArgs` (`{denixLibName, ...}: denixLibName.module { ... }`). Defaults to `delib` ; changes are not recommended.
- `extensions` (listOf delib.extension): a list of extensions to be applied to the configuration. Defaults to `[]` .
- `nixpkgs` (nixpkgs): used to override nixpkgs in your configuration, equivalent to `inputs.denix.inputs.nixpkgs.follows` . Defaults to `inputs.nixpkgs` .
- `home-manager` (home-manager): used to override home-manager in your configuration, equivalent to `inputs.denix.inputs.home-manager.follows` . Defaults to `inputs.home-manager` .
- `nix-darwin` (nix-darwin): used to override nix-darwin in your configuration, equivalent to `inputs.denix.inputs.nix-darwin.follows` . Defaults to `inputs.nix-darwin` .
- `homeManagerNixpkgs` (nixpkgs): used in the `pkgs` attribute of the `home-manager.lib.homeManagerConfiguration` function in the format: `homeManagerNixpkgs.legacyPackages.${host :: homeManagerSystem}` . Defaults to the `nixpkgs` provided in the function arguments.
- `useHomeManagerModule` (boolean): whether to include the Home Manager module in the NixOS and Nix-Darwin configurations. Defaults to `true` ; can be overridden per host via `delib.host :: useHomeManagerModule` .
- `homeManagerUser` (string): the username, used in `home-manager.users.`

`${homeManagerUser}` and for generating the Home Manager configuration list. Can be overridden per host via `delib.host :: homeManagerUser`.

- `moduleSystem` ("nixos", "home", and "darwin"): specifies which module system the configuration list should be generated for - NixOS, Home Manager, or Nix-Darwin.
- `paths` (listOf string): paths to be imported; add hosts, rices, and modules here. Defaults to `[]`.
- `exclude` (listOf string): paths to be excluded from importing. Defaults to `[]`.
- `recursive` (boolean): determines whether to recursively search for paths to import. Defaults to `true`.
- `specialArgs` (attrset): `specialArgs` to be passed to `lib.nixosSystem`, `home-manager.lib.homeManagerConfiguration`, and `nix-darwin.lib.darwinSystem`. Defaults to `{}`.
- **EXPERIMENTAL** `extraModules` (list): defaults to `[]`.
- **EXPERIMENTAL** `mkConfigurationsSystemExtraModule` (attrset): a module used in the internal NixOS configuration that receives the list of hosts and rices to generate the configuration list. Defaults to `{nixpkgs.hostPlatform = "x86_64-linux";}`.

Pseudocode

nix

```nix
delib.configurations rec {
  myconfigName = "myconfig";
  denixLibName = "delib";
  extensions = with delib.extensions; [];
  nixpkgs = inputs.nixpkgs;
  home-manager = inputs.home-manager;
  nix-darwin = inputs.nix-darwin;
  homeManagerNixpkgs = nixpkgs;
  useHomeManagerModule = true;
  homeManagerUser = "sjohn";
  moduleSystem = "nixos";
  paths = [./modules ./hosts ./rices];
  exclude = [./modules/deprecated];
  recursive = true;
  specialArgs = {
```

```
    inherit inputs;
  };
}
```

Edit this page on Github

Last updated: 8/5/25, 4:09 PM

Previous page
Introduction

Next page
Introduction

Introduction to Denix Extensions

Some features that contradict the philosophy of Denix or are purely optional can still be implemented as separate extensions. Also, if a configuration needs to provide a custom function or modify the behavior of an existing one - this can be done through a custom extension.

Currently, the extension API allows you to:

- Extend `delib` by adding new functions or overriding existing ones.
- Add modules that are automatically included in the configuration.

Usage

To add extensions, you need to pass the `extensions` argument to the `delib.configurations` function. This argument is a list of extensions.

Official extensions are located in `delib.extensions` (see All extensions). In addition to them, you can also add custom extensions in the following ways:

- If the extension is a single file:

```nix
delib.callExtension ./myExtension.nix
```

- If the extension consists of multiple files:

```nix
delib.mergeExtensions "myExtensionName" [
  (delib.callExtension ./myExtensionPart1.nix)
  (delib.callExtension ./myExtensionPart2.nix)
```

```
]
```

- If there are many extensions:

```nix
(delib.callExtensions { paths = [./myExtensions]; }).myExtension
```

Some extensions support their own configuration (not to be confused with module options). Configuration is done via `withConfig` :

```nix
delib.extensions.someExtension.withConfig {
  someOption = "value";
}
```

The parameter can be either an overlay or an attrset (it will automatically be converted to an overlay). For details, see NixOS Wiki - Overlays.

Example

```nix
delib.configurations {
  # ...
  extensions =
    with delib.extensions;
    with delib.callExtensions { paths = [./myExtensions]; };
    [
      args
      (base.withConfig (final: prev: {
        args.enable = true;
        hosts.type.types = prev.hosts.type.types ++ ["laptop"];
      }))
      (delib.callExtension ./myExtension.nix)
      extensionFromMyExtensions
      (anotherExtensionFromMyExtensions.withConfig { someConfig = "someValue"
    ];
}
```

Edit this page on Github

Last updated: 7/8/25, 1:07 PM

Previous page
Structure

Next page
Structure

Structure

Each extension is created using `delib.extension`.

Function arguments

- `name`: string. Must be unique. Used to look up the extension in `delib.extensions` or `delib.callExtensions`.
- `description`: string. A short description. Defaults to `""`, but is required for official extensions.
- `maintainers`: a list of maintainers responsible for the extension. Defaults to `[]`.
- `config`: overlay. If a lambda with 1 parameter or an attrset is provided, it will be converted into a lambda with 2 parameters using `nixpkgs.lib.toExtension`, where the lambda with 1 parameter is treated as `prev`. Defaults to `final: prev: {}`.
- `initialConfig`: an object to which overlays are applied, or null. If an attrset is provided, it will be converted into a lambda with 1 parameter `final`. If you merge extensions, only one of them may have a non-null `initialConfig`. Defaults to `null`.
- `configOrder`: integer. Specifies the order in which the `config` of this extension is composed with the `config` of other extensions when merged. The smaller the number, the earlier it is applied. Defaults to `0`.
- `libExtension`: an overlay with the extension's configuration. If a lambda with 1 parameter is provided, it is `config`; if two - `config` and `prev`. Applied to `delib`. Defaults to `config: final: prev: {}`.
- `libExtensionOrder`: integer. Specifies the order in which the `libExtension` of this extension is composed with the `libExtension` of other extensions when merged. The smaller the number, the earlier it is applied. Defaults to `0`.
- `modules`: a list of modules with the extension's configuration. If a lambda with 1

parameter is provided, that parameter is `config`. The listed modules will be imported into configurations using this extension. It is strongly discouraged to use `delib` from the arguments of the file where the extension is defined when declaring modules. Instead, use `delib` inside the modules themselves: `modules = [({delib, ...}: delib.module {/* ... */})]`. Defaults to `[]`.

Pseudocode

nix

```nix
delib.extension {
name = "test";
description = "";
maintainers = with delib.maintainers; [sjohn];

# config = {};
# config = prev: {};
config = final: prev: {};

# initialConfig = {};
# initialConfig = final: {}
initialConfig = null;

configOrder = 0;

# libExtension = {};
# libExtension = config: {};
# libExtension = config: prev: {};
libExtension = config: final: prev: {};

libExtensionOrder = 0;

# modules = config: [];
modules = [];
}
```

Edit this page on Github

Last updated: 7/8/25, 1:07 PM

Previous page
Introduction

Next page
Development

Denix Documentation

Menu On this page

Development

This section describes how to properly import extensions. For examples of creating extensions, study the source code of existing extensions.

Importing

First, decide how many extensions and files you want to import.

If you have multiple extensions, or they are split across files:

nix

```nix
delib.callExtensions {
  paths = [./foo ./default.nix]; # default: []
  exclude = [./foo/test.nix];    # default: []
  recursive = true;              # default: true
}
# Returns an attrset where the keys are extension names and the values are th
```

If you have a single extension represented by one file:

nix

```nix
delib.callExtension ./default.nix
# Returns a single extension.
```

Distributing

To make your extensions available for use in other flakes, add the `denixExtensions`

attrset to `outputs`:

nix

```nix
outputs = {denix, ...}: {
  denixExtensions = denix.lib.callExtensions {
    paths = [./extensions];
  };
};
```

You can also create a separate flake that contains only extensions:

flake.nix

nix

```nix
{
  description = "Denix extensions collection";

  inputs.denix.url = "github:yunfachi/denix";

  outputs =
    { denix, ... }:
    {
      denixExtensions = denix.lib.callExtensions {
        paths = [ ./extensions ];
      };
    };
}
```

You can use a template for this:

sh

```sh
nix flake init -t github:yunfachi/denix#extensions-collection
```

If you want to use `denixExtensions` in the same flake that contains your configuration, refer to `self`:

nix

```nix
outputs = {denix, self, ...}: let
  mkConfigurations = moduleSystem:
```

```
    # ...
    denix.lib.configurations {
      extensions = with denix.lib.extensions; with self.denixExtensions; [
        args # denix.lib.extensions
        awesomeExtension # self.denixExtensions
      ];
    };
in {
  denixExtensions = denix.lib.callExtensions {paths = [./extensions];};
};
```

Edit this page on Github

Last updated: 7/8/25, 1:07 PM

Previous page
Structure

Next page
All Extensions

All Extensions

This section describes all official Denix extensions.

Args

Name	`args`
Description	More convenient way to configure `_module.args` via `myconfig`
Maintainers	yunfachi (GitHub, Telegram)

Settings

Name	Default Value	Description
`path`	`"args"`	Path in `myconfig` where options will be created

Base

Name	`base`
Description	Creates feature-rich and fine-tunable modules for hosts and rices with minimal effort
Maintainers	yunfachi (GitHub, Telegram)

Settings

Name	Default Value	Description
`enableAll`	`true`	Default value for `hosts.enable` and `rices.enable`

`args`

Name	Default Value	Description
`enable`	`false`	Default value for `hosts.args.enable` and `rices.args.enable`
`path`	`"args"`	Default value for `hosts.args.path` and `rices.args.path`

`assertions`

Name	Default Value	Description
`enable`	`true`	Default value for `hosts.assertions.enable` and `rices.assertions.enable`
`moduleSystem`	`"home-manager"`	Default value for `hosts.assertions.moduleSystem` and `rices.assertions.moduleSystem`

`hosts`

Name	Default Value	Description
`enable`	`enableAll`	Whether to create the hosts module
`extraSubmodules`	`[]`	Additional submodules used to define custom options within the host type. Example value: `[ ({ config, ... }: { options.coolName = "cool " + config.name; }) ]`

`hosts.args`

Name	Default Value	Description
`enable`	`args.enable`	Whether to create `host` and `hosts` arguments with the `args` extension
`path`	`args.path`	Path to options of the `args` extension

`hosts.system`

Name	Default Value	Description
`enable`	`true`	Whether to create a string option `system` in the host submodule, which sets the values for the `homeManagerSystem`, `nixos.nixpkgs.hostPlatform`, and `darwin.nixpkgs.hostPlatform` options

`hosts.assertions`

Name	Default Value	Description
`enable`	`assertions.enable`	Whether to add `delib.hostNamesAssertions` to the `assertions` option of the specified module system
`moduleSystem`	`assertions.moduleSystem`	Module system to which assertions will be added. Any value is allowed, even `"myconfig"` if you have a dedicated module for that. Values `"home-manager"` and `"nix-darwin"` will be automatically converted to `"home"` and `"darwin"` respectively

`hosts.type`

Name	Default Value	Description
`enable`	`true`	Whether to create an enum option `type` in the host submodule
`generateIsType`	`true`	Whether to generate a boolean option for each value in `hosts.type.types`, formatted as `"is{Type}"`. Also creates `"isPC"`, with a value of `isDesktop || isLaptop`
`types`	`["desktop" "laptop" "server"]`	All allowed values for the `type` option in the host submodule. Note: to append to the list rather than overwrite it, use: `types = prev.types ++ ["newType"];`

`hosts.features`

Name	Default Value	Description
`enable`	`true`	Whether to create the `features` option with an enum-list type in the host submodule
`generateIsFeatured`	`true`	Whether to generate a boolean option for each value in `hosts.features.features`, formatted as `"{feature}Featured"`
`features`	`[]`	All allowed values for the `features` option in the host submodule
`default`	`[]`	Default value for the `features` option in the host submodule
`defaultByHostType`	`{}`	Attrset where keys are one of the `hosts.type.types` values and values are lists similar to `hosts.features.default`. Merged with `hosts.features.default` and the list whose key matches the value of the `type` option in the host submodule

`hosts.displays`

Name	Default Value	Description
`enable`	`true`	Whether to create the `displays` option in the host submodule. Its type is a list of submodules with the following options: `enable` (bool; default `true`), `name` (str; no default), `primary` (bool; default `true`, but only if the `displays` options in the host submodule contains one item), `touchscreen` (bool; default `false`), `refreshRate` (int; default `60`), `width` (int; default `1920`), `height` (int; default `1080`), `x` (int; default `0`), `y` (int; default `0`)

`rices`

Name	Default Value	Description
`enable`	`enableAll`	Whether to create the rices module
`extraSubmodules`	`[]`	Additional submodules used to define custom options within the rice type. Example value: `[ ({ config, ... }: { options.coolName = "cool " + config.name; }) ]`

`rices.args`

Name	Default Value	Description
`enable`	`args.enable`	Whether to create `rice` and `rices` arguments using the `args` extension
`path`	`args.path`	Path to options of the `args` extension

> `rices.assertions`
>
> | Name | Default Value | Description |
> | --- | --- | --- |
> | `enable` | `assertions.enable` | Whether to add `delib.riceNamesAssertions` to the `assertions` option of the specified module system |
> | `moduleSystem` | `assertions.moduleSystem` | Module system to which assertions will be added. Any value is allowed, even `"myconfig"` if you have a dedicated module for that. `"home-manager"` and `"nix-darwin"` will be automatically converted to `"home"` and `"darwin"` respectively |

User

WIP

Overlays

Name	`overlays`
Description	Simplified overlay configuration module
Maintainers	Zonni (GitHub)

Settings

Name	Default Value	Description
`defaultTargets`	`["nixos"`	Default targets to apply overlays

Name	Default Value	Description
	`"home"]`	
`moduleNamePrefix`	`"overlays"`	Prefix added to module names when `withPrefix` is true

Library

`overlayModule`

Creates a module that applies overlays for specified targets. If `withPrefix` is true, it creates the option `${moduleNamePrefix}.${name}.enable` with the value `delib.overlayModule :: enabled` ; otherwise, it does the same but in `${name}.enable` .

Arguments:

Name	Type	Default Value	Description
`name`	string	*required*	Name of the overlay module
`overlay`	overlay function	`null`	Single overlay function to apply
`overlays`	list of overlay functions	`[]`	List of overlay functions to apply
`targets`	list of strings	`defaultTargets`	Target systems to apply overlays to. Valid values: `"nixos"` , `"home"` , `"darwin"`
`withPrefix`	boolean	`true`	Whether to add `moduleNamePrefix` to the module name
`enabled`	boolean	`true`	Whether overlay should be enabled by default

Edit this page on Github

Last updated: 8/30/25, 8:36 AM

Previous page
Development

Next page
Introduction

Introduction to Denix Rices

A rice is an attribute set returned by the function `delib.rice`, which enables or disables specific configurations depending on the value of the option `${myconfigName}.rice`. It also passes the input attribute set of the `delib.rice` function to the `${myconfigName}.rice.${delib.rice :: name}` option.

The term "rice" in slang refers to system settings, usually related to appearance. In the Denix context, a rice represents a configuration that can be applied **to any** host and will function correctly.

However, rices are not mandatory: to avoid using them, simply do not add the options `${myconfigName}.rices` and `${myconfigName}.rice`, and do not use the `delib.rice` function.

Inheritance

A rice can inherit all configurations of another rice via the `inherits` attribute. Additionally, you can set `inheritanceOnly = true;`, which will hide the rice from being generated in `delib.configurations`, leaving it only for inheritance.

Example of three rices, where the first two inherit all configurations from the "rounded" rice:

nix

```nix
{delib, ...}:
delib.rice {
  name = "black";
  inherits = ["rounded"];
  myconfig.colorscheme = "black";
```

```
}
```

nix

```nix
{delib, ...}:
delib.rice {
  name = "light";
  inherits = ["rounded"];
  myconfig.colorscheme = "light";
}
```

nix

```nix
{delib, ...}:
delib.rice {
  name = "rounded";
  inheritanceOnly = true;
  myconfig.hyprland.rounding = "6";
}
```

Options

For rices to work, your configuration must include the options `${myconfigName}.rice` and `${myconfigName}.rices`, which **you define yourself** in one of the modules.

Example of a minimal recommended configuration for rices:

nix

```nix
{delib, ...}:
delib.module {
  name = "rices";

  options = with delib; let
    rice = {
      options = riceSubmoduleOptions;
    };
  in {
    rice = riceOption rice;
    rices = ricesOption rice;
```

```
    };

    home.always = {myconfig, ...}: {
      assertions = delib.riceNamesAssertions myconfig.rices;
    };
  }
```

More examples can be found in the Examples section.

Edit this page on Github

Last updated: 10/9/24, 11:51 AM

Previous page
All Extensions

Next page
Structure

Structure

Function Arguments

- `name` : a string representing the rice name.
- `inherits` : a list of strings - the names of rices whose configurations will be inherited by the current rice.
- `inheritanceOnly` : a boolean value that determines whether this rice will be added to the list of systems generated for each host and rice, or if it is only used for inheritance.
- `myconfig` : sets its value to `config.${myconfigName}` if `config.${myconfigName}.rice` matches the current rice.
- `nixos` : sets its value to `config` if `moduleSystem` is `nixos` and `config.${myconfigName}.rice` matches the current rice.
- `home` : sets its value to `config` if `moduleSystem` is `home` and `config.${myconfigName}.rice` matches the current rice. Otherwise, if `config.${myconfigName}.rice` matches the current rice, sets its value to `config.home-manager.users.${homeManagerUser}` .
- `darwin` : sets its value to `config` if `moduleSystem` is `darwin` and `config.${myconfigName}.rice` matches the current rice.

Passed Arguments

A list of arguments passed to `[myconfig|nixos|home|darwin]` if their type is `lambda` :

- `name` : the same `name` as in the arguments of `delib.rice` .

- `myconfig` : equals `config.${myConfigName}` .
- `cfg` : equals `config.${myConfigName}.rices.${delib.rice :: name}` .

Pseudocode

nix

```nix
delib.rice {
  name = "";

  inherits = [];

  inheritanceOnly = [];

  # if config.${myconfigName}.rice == name
  # then {config.${myConfigName} = ...;}
  # else {}
  myconfig = {name, cfg, myconfig, ...}: {};

  # if config.${myconfigName}.rice == name, then
  #   if moduleSystem == "nixos"
  #   then {config = ...;}
  #   else {}
  # else {}
  nixos = {name, cfg, myconfig, ...}: {};

  # if config.${myconfigName}.rice == name, then
  #   if moduleSystem == "home"
  #   then {config = ...;}
  #   else {config.home-manager.users.${homeManagerUser} = ...;}
  # else {}
  home = {name, cfg, myconfig, ...}: {};

  # if config.${myconfigName}.rice == name, then
  #   if moduleSystem == "darwin"
  #   then {config = ...;}
  #   else {}
  # else {}
```

```
  darwin = {name, cfg, myconfig, ...}: {};
}
```

Edit this page on Github

Last updated: 5/20/25, 11:28 AM

Previous page
Introduction

Next page
Examples

Denix Documentation

Menu On this page

Examples

Minimally Recommended Rice Module:

An example of a minimal rice module configuration that serves as a baseline for all further settings:

nix

```nix
{delib, ...}:
delib.module {
  name = "rices";

  options = with delib; let
    rice = {
      options = riceSubmoduleOptions;
    };
  in {
    rice = riceOption rice;
    rices = ricesOption rice;
  };

  home.always = {myconfig, ...}: {
    assertions = delib.riceNamesAssertions myconfig.rices;
  };
}
```

Short Version

Using `delib.riceNamesAssertions` is strongly recommended, but it can also be omitted.

nix

```nix
{delib, ...}:
delib.module {
  name = "rices";

  options = with delib; let
    rice.options = riceSubmoduleOptions;
  in {
    rice = riceOption rice;
    rices = ricesOption rice;
  };
}
```

Edit this page on Github

Last updated: 10/9/24, 11:51 AM

Previous page
Structure

Next page
Common Errors

Denix Documentation

Menu On this page

Common Errors

This section lists some common errors and their possible solutions.

Conditional Imports

If for some reason you need to perform an import inside the Denix module, then attempting to set `[nixos|home|darwin].[ifEnabled|ifDisabled].imports` will cause an `infinite recursion` error.

This means that if you need to import, for example, a third-party library, you should first add it to `[nixos|home|darwin].always.imports`, and then, if necessary, set option values in `[nixos|home|darwin].[always|ifEnabled|ifDisabled]`.

Discussion on "Conditional module imports": discourse.nixos.org

Incorrect Code

nix

```nix
{delib, ...}:
delib.module {
  name = "wrong";
  options = delib.singleEnableOption true;

  nixos.ifEnabled = {
    imports = [<someModule>];

    someModuleOptions.enable = true;
  };
}
```

Correct Code

nix

```nix
{delib, ...}:
delib.module {
  name = "correct";
  options = delib.singleEnableOption true;

  nixos.always.imports = [<someModule>];

  nixos.ifEnabled.someModuleOptions.enable = true;
}
```

Edit this page on Github

Last updated: 5/20/25, 11:28 AM

Previous page
Examples

Next page
Showcase

Denix Documentation

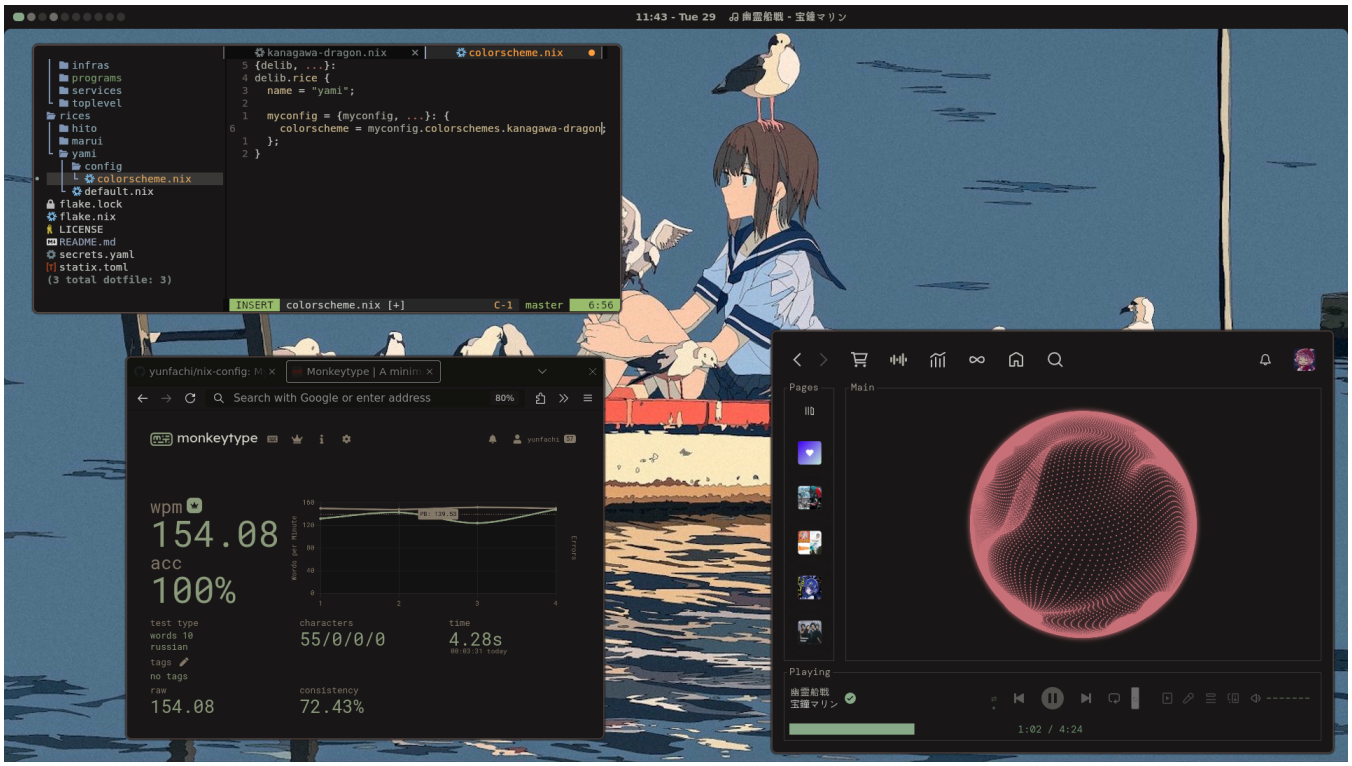
Menu

On this page

Showcase

This section presents a list of NixOS, Home Manager, and Nix-Darwin configurations that use Denix and can serve as examples of good practice*. You can add your configuration to this list via a pull request or an issue.

Yunfachi - nix-config (NixOS, Home Manager)



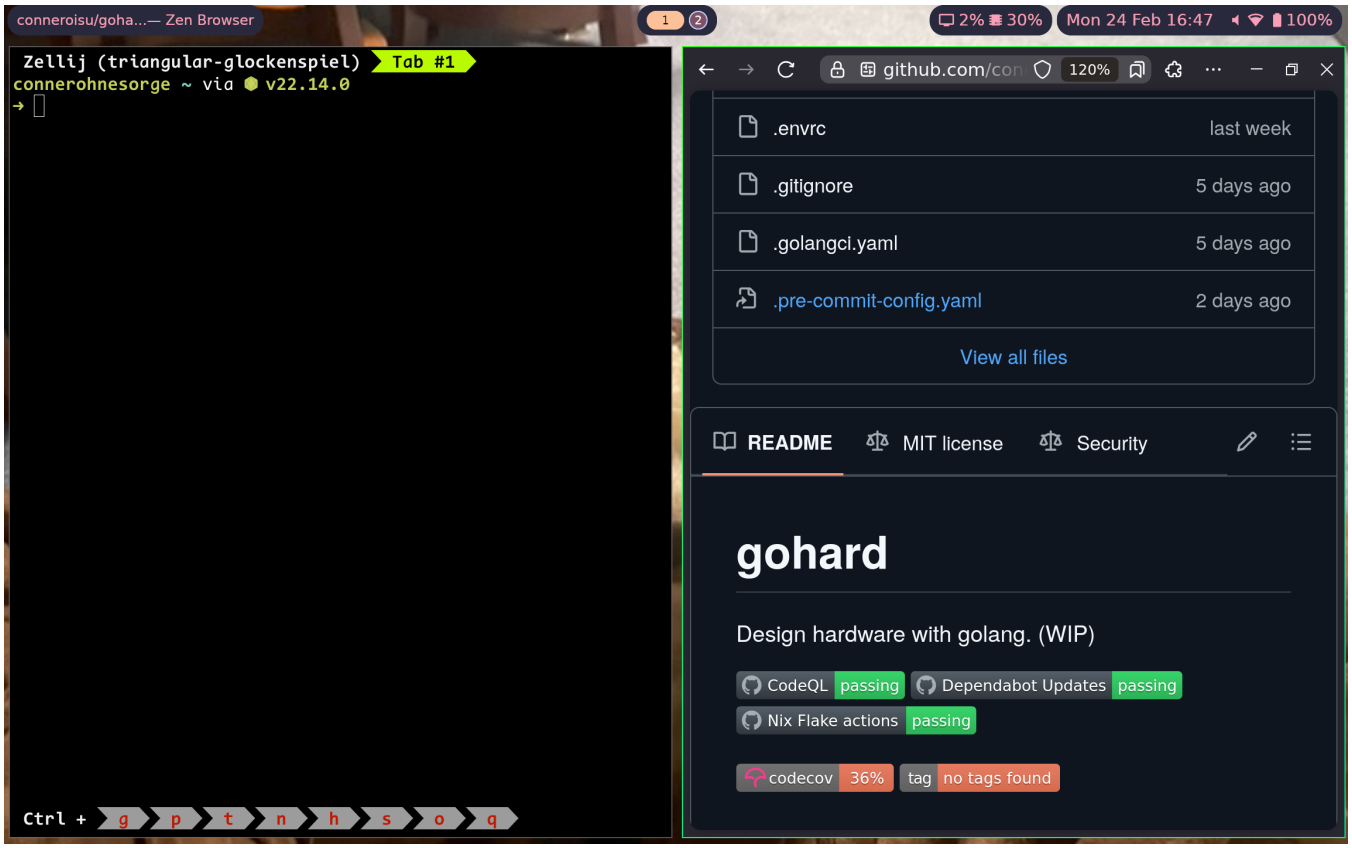
Features:

- Extracting relevant options from modules into host options
- Module for specifying `_module.args`
- Modules for different infrastructures

- Shared and host-specific secrets using SOPS
- Color scheme-based rices inheriting general configuration settings

---

Conneroisu - dotfiles (NixOS, Home Manager, Nix-Darwin)



Features:

- NixOS and Nix-Darwin configurations
- Custom packages
- Determinate Nix support

Edit this page on Github

Last updated: 6/28/25, 9:33 AM

Previous page
Common Errors